3 Grammaire hors contextes

1 Nettoyage de grammaires

1.1 Introduction

Regardons rapidement la grammaire suivante :

$S \to AB \quad S \to CB$
$A \to \varepsilon \quad A \to Aa$
$B \to \varepsilon \quad B \to bB$
$C \to cDS \quad B \to cdES$
$D \to dC$
$E \to C$
$F \to CD \quad F \to CG$
$G \to CF$

Cette grammaire est hors contexte généralisée. Mais on peut voir différentes choses :

- les symboles F et G ne peuvent faire partie d'une forme sententielle dérivant de S, on dit qu'ils sont inaccessibles,
- de plus, en regardant de plus près, on se rend compte qu'on ne peut pas créer un terminal dès lors qu'on a le symbole C, D ou E. On dit qu'un tel symbole est improductif.

On aimerait pouvoir distinguer les grammaires qui présentent ce genre de problème des autres grammaires. Nous en arrivons donc à la notion de **grammaire propre**.

1.2 Grammaire propre

Définition 1.1 Une grammaire $G = (N, T, P, S)$ est dite **propre** lorsque :

- si $X \to \varepsilon \in P$ alors $X = S$ et S ne figure pas en partie droite de règle.
- $\forall X \ \in N, L_X(G) \neq \emptyset$
- G n'admet aucune règle de la forme $X \to Y$ avec $Y \in N$.
- $\forall X \ \in V$, il existe un mot $w \in V^+$ contenant le symbole X et tel que $S \overset{*}{\vdash} w$

Théorème 1.1 Tout langage hors-contexte non vide L admet une grammaire hors-contexte propre.

La preuve de ce théorème tient dans le fait qu'on peut « nettoyer » une grammaire hors-contexte.

1.3 Nettoyage de grammaire

Le nettoyage de grammaire se fait en 4 étapes (à réaliser dans cet ordre) :

- supprimer les symboles inutiles *(étape optionnelle, mais permettant de diminer significativement la taille de la grammaire pour les étapes suivantes)*,
- rendre la grammaire ε-libre,
- supprimer les productions simples,
- supprimer les symboles inutiles (étape nécessaire, même si on l'a déjà faite au début).

Rendre une grammaire ε-libre

On a pu voir plusieurs fois le côté pratique des règles de type $A \to \varepsilon$. Il s'agit ici de les supprimer, en se ramenant à la forme standard, où $A \to \varepsilon \in P$ implique que $A = S$ et que S n'apparaît pas en partie droite de règle.

Passage à une grammaire ε-libre

Entrée:
| $G = (N, T, P, S)$; // une grammaire hors-contexte
Sortie:
| $G' = (N', T, P', S')$; // équivalente à G et ε-libre
Réalisation:
| // Construire $N_\varepsilon = \{A \in N | A \overset{+}{\vdash} \varepsilon\}$
| $N_0 \longleftarrow \emptyset$; $i \longleftarrow 0$;
| **répéter**
| | $i \longleftarrow i + 1$;
| | $N_i \longleftarrow N_{i-1} \cup \{A \in N | A \to \alpha \in P \wedge \alpha \in N_{i-1}^*\}$;
| **jusqu'à** $N_i = N_{i-1}$;
| $N_\varepsilon \longleftarrow N_i$;
| // Construire P' en remplaçant en partie droite toutes les occurrences d'un symbole de N_ε par lui-même ou par ε (sans introduire de règle-ε).
| $P' \longleftarrow \emptyset$;
| **pour chaque** *règle* $A \to \alpha_0 B_1 \alpha_1 \dots \alpha_{p-1} B_p \alpha_p \in P$ *avec* $p > 0 \wedge B_i \in N_\varepsilon \wedge \alpha_i \in (V \backslash N_\varepsilon)^*$
| | ajouter à P' les règles $A \to \alpha_0 X_1 \alpha_1 \dots \alpha_{p-1} X_p \alpha_p$ où $X_i \in \{B_i, \varepsilon\}$ (sauf éventuellement $A \to \varepsilon$)
| **pour chaque** *règle* $A \to \alpha$ *avec* $\alpha \in (V \backslash N_\varepsilon)^+$
| | ajouter à P' la règle $A \to \alpha$ (non modifiée)
| **si** $S \notin N_\varepsilon$
| | $N' \longleftarrow N$; $S' \longleftarrow S$;
| **sinon**
| | // Créer un nouveau symbole S'
| | $N' \longleftarrow N \cup \{S'\}$;
| | ajouter à P' les règles $S' \to S$ et $S' \to \varepsilon$;
| $\to G = (N', T, P', S')$;

Suppression des productions simples

Il s'agit de supprimer toutes les productions de forme $A \to B$ (appelées **productions simples**), ce qui, du même coup, produit une grammaire **sans cycles** [1] :

Suppression des productions simples

Entrée:
| $G = (N, T, P, S)$; // une grammaire hors contexte
Sortie:
| $G' = (N, T, P', S)$; // équivalente à G et sans règles simples. Si G est ε-libre, G' l'est aussi.
Réalisation:
| // Construire les $N_A = \{B \in N | A \overset{*}{\vdash} B\}$
| **pour chaque** $A \in N$
| | $N_0 \longleftarrow \{A\}$; $i \longleftarrow 0$;
| | **répéter**
| | | $i \longleftarrow i + 1$;
| | | $N_i \longleftarrow N_{i-1} \cup \{C \in N | B \to C \in P \wedge B \in N_{i-1}\}$;
| | **jusqu'à** $N_i = N_{i-1}$;
| | $N_A \longleftarrow N_i$;
| // Construire P'
| $P' \longleftarrow \emptyset$;
| **pour chaque** *règle non simple* $A \to \alpha \in P$
| | **pour chaque** B *tel que* $A \in N_B$
| | | ajouter à P' la règle $B \to \alpha$
| $\to G = (N, T, P', S)$;

Suppression des symboles inutiles

Un symbole $X \in V$ est dit **inutile** s'il est soit improductif, soit inaccessible.
Un symbole $A \in N$ est **improductif** ssi $L_A(G) = \emptyset$: A ne dérive vers aucune chaîne terminale.
Un symbole $X \in V$ est **inaccessible** ssi aucune forme sententielle ne contient X :

$$(\forall \gamma\ , \delta \in V^*)[\neg(S \overset{*}{\vdash} \gamma X \delta)]$$

1. Une grammaire est sans cycle ssi $\neg(\exists A \in N, A \overset{+}{\vdash} A)$

Pour supprimer les symboles inutiles, il faut commencer par supprimer les symboles improductifs (ce qui peut introduire des inaccessibles). Ensuite seulement, on supprimera les inaccessibles.

Suppression des improductifs On construit l'ensemble des symboles productifs et on ne garde que les règles sans improductifs.

Suppression des symboles improductifs

Entrée:
$G = (N, T, P, S)$; // une grammaire hors contexte
Sortie:
$G_p = (N_p, T, P_p, S)$; // équivalente à G et sans symbole improductif
Réalisation:
// Construire N_p ensemble des symboles productifs
$N_0 \longleftarrow \emptyset$; $i \longleftarrow 0$;
répéter
$i \longleftarrow i + 1$;
$N_i \longleftarrow N_{i-1} \cup \{A \in N | A \to \alpha \in P \wedge \ \alpha \in (N_{i-1} \cup T)^*\}$;
jusqu'à $N_i = N_{i-1}$;
$N_p \longleftarrow N_i$;
si $S \notin N_p$ **déclencher exception** *langage vide*;
;
// Construire P_p sans les règles utilisant un improductif
$P_p \longleftarrow \{A \to \alpha \in P | A \in N_p \wedge \ \alpha \in (N_p \cup T)^*\}$;
$\to G = (N_p, T, P_p, S)$;

Suppression des inaccessibles On construit l'ensemble des symboles (terminaux et non-terminaux) accessibles à partir de S et on ne garde que les règles sans inaccessibles.

Suppression des symboles inaccessibles

Entrée:
$G = (N, T, P, S)$; // une grammaire hors contexte
Sortie:
$G_p = (N', T', P', S)$; // équivalente à G et sans symbole inaccessible
Réalisation:
// Construire V_a ensemble des symboles (terminaux et non-terminaux) accessibles
$V_0 \longleftarrow S$; $i \longleftarrow 0$;
répéter
$i \longleftarrow i + 1$;
$V_i \longleftarrow V_{i-1} \cup \{X \in V | \exists A \to \alpha X \beta \in P \wedge \ A \in V_{i-1}$;
jusqu'à $V_i = V_{i-1}$;
$V_a \longleftarrow V_i$;
// On supprime de N et T les symbole inaccessible
$N' \longleftarrow N \cap V_a$;
$T' \longleftarrow T \cap V_a$;
// On supprime de P les productions contenant un symbole inaccessible
$P' \longleftarrow \{A \to \alpha \in P | A \in N' \wedge \ \alpha \in V'^*\}$;
$\to G = (N', T', P', S)$;

2 Forme normale de Chomsky, algorithme de Cocke

Maintenant que nous savons déterminer si un langage est sous-contexte ou hors-contexte, nous pouvons nous s'atteler à la tâche qui nous intéresse vraiment : l'analyse de langages hors-contexte.
Pour cela, nous verrons que, sans perdre de puissance d'expression, on peut se donner des contraintes supplémentaires sur la forme des règles hors-contexte, cela afin de mettre en œuvre des algorithmes d'analyse fonctionnant sur l'ensemble des langages hors-contexte.

2.1 Forme normale de Chomsky

On dit qu'une grammaire hors-contexte est sous forme normale de Chomsky (FNC) ssi toutes les productions sont de l'une des formes suivantes :

$A \to a$	(règle terminale)
$A \to BC$	(règle binaire)
$S \to \varepsilon$	(mais alors S ne figure dans aucune partie droite)

Cette forme peut sembler très contraignante, mais nous prouverons plus tard le théorème aussi simple que fondamental suivant : tout langage hors-contexte possède une grammaire sous forme normale de Chomsky.

Exemple E3.15:

G : $S \to a$ $S \to SX$ $S \to TY$ $S \to PZ$
$T \to a$ $T \to TY$ $T \to PZ$
$F \to a$ $F \to PZ$
$X \to AT$ $P \to$ '(' $A \to$ '+'
$Y \to MF$ $Q \to$ ')' $M \to$ '*'
$Z \to SQ$

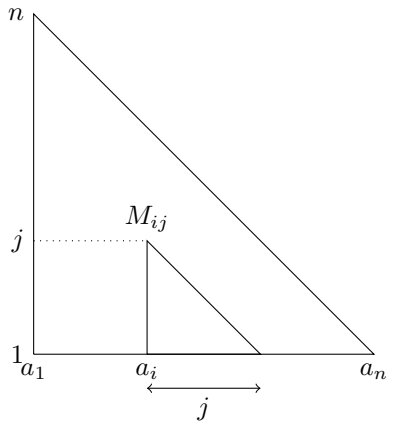
2.2 Passage en Forme Normale de Chomsky

Toute grammaire hors-contexte possède une grammaire équivalente sous forme normale de Chomsky. La démonstration de ce théorème tient dans le fait qu'il existe un algorithme permettant de transformer une grammaire propre en une grammaire FNC, tout en conservant l'équivalence.

Passage en Forme Normale de Chomsky

Entrée:
$G = (N, T, P, S)$; // une grammaire hors-contexte propre
Sortie:
$G' = (N', T, P', S')$; // équivalente à G et en FNC
Réalisation:
// Initialement, P' est vide, on construit les règles de P' à partir de celle de P en créant éventuellement de nouveaux non terminaux
$P' \longleftarrow \emptyset$; $N' \longleftarrow \emptyset$;
mettre dans P' toutes les règles $A \to a \in P$ telles que $A \in N$ et $a \in T$;
mettre dans P' toutes les règles $A \to BC \in P$ telles que $A, B, C \in N$;
mettre dans P' la règle $S \to \varepsilon$ si elle est présente dans P;
pour chaque *règle* $A \to X_1 X_2 \dots X_k \in P$ *avec* $k > 2 \wedge\ X_i \in V$
  ajouter à P' les règles suivantes :
  $A \to X'_1 < X_2 \dots X_k >$
  $< X_2 \dots X_k > \to X'_2 < X_3 \dots X_k >$
  ...
  $< X_{k-1} \dots X_k > \to X'_{k-1} X'_k$
  où $X'_i = X_i$ si $X_i \in N$;
  Ajouter à N' chaque symbole X'_i et $< Xi \dots X_k >$ $(2 \geq i \geq k-1)$ nouvellement créé;
pour chaque *règle* $A \to X_1 X_2 \in P$ *avec* X_1 *ou* $X_2 \in T$
  ajouter à P' la règle $A \to X'_1 X'_2$ (avec les même conventions que ci dessus)
pour chaque *symbole* X'_i *nouvellement créé*
  ajouter $X'_i \to X_i$ à P;
$\to G = (N', T, P', S')$;

2.3 Algorithme de Cocke



Cette forme normale présente un avantage : tous les arbres de dérivations sont des arbres binaires. C'est aussi cette forme normale qui nous permet d'utiliser l'algorithme de Cocke que nous allons voir maintenant. À noter que cet algorithme est un algorithme général, qui prend en paramètre la grammaire et le mot à reconnaître.

Son principe assez astucieux :

Soit $x = a_1 \dots a_n$ le mot à analyser.

Si $x = \varepsilon$, $x \in L(G)$ ssi $S \to \varepsilon \in P$.

Sinon, on construit un tableau triangulaire M où $M_{ij} = \{A | A \overset{*}{\vdash} a_i \dots a_{i+j-1}\}$: les symboles contenus dans M_{ij} sont exactement ceux qui peuvent dériver vers la sous-chaîne de x commençant

en $i^{\text{ème}}$ position et de longueur j. On remplit ce tableau en partant du bas (par les cases dominant les sous-chaînes les plus courtes).

Une fois ce tableau rempli, on sait, pour chaque sous-chaîne du mot à analyser, quels sont les non-terminaux qui peuvent le générer dans la grammaire. Il suffit donc de regarder si la case M_{1n} contient le non-terminal S.

Cet algorithme est dû (apparemment indépendamment) à Cocke, Younger et Kasami. Il est aussi connu sous le nom d'algorithme de Cocke-Younger-Kasami (CYK).

Algorithme de Cocke

Entrée:
| $G = (N, T, P, S)$ une grammaire FNC; // grammaire d'entrée sous forme normale de Chomsky
| $x = a_1 \dots a_n \in T (n > 0)$; // mot à reconnaître
Sortie:
| vrai ssi $x \in L(G)$
Lexique:
| M : un tableau sur $[1..n, 1..n]$ de 2^N ; // tableau d'ensembles de non-terminaux
Réalisation:
| **si** $x = \varepsilon$
| | $\rightarrow (S \rightarrow \varepsilon \in P)$
| **pour** i **parcourant** *[1..n]* // remplissage de la première ligne par les règles unaires.
| | *On « réduit » les terminaux*;
| | $M_{i,1} \longleftarrow \{A \in N | A \rightarrow a_i \in P\}$;
| **pour** j **parcourant** *[2..n]*
| | **pour** i **parcourant** *[1..n-j+1]*
| | | **pour** k **parcourant** *[1..j-1]*
| | | | $M_{i,j} \longleftarrow M_{i,j} \cup \{A \in N | A \rightarrow BC \in P \wedge\ B \in M_{i,k} \wedge\ C \in M_{i+k,j-k}\}$;
| $\rightarrow (S \in M_{1,n})$;

Intérêt de l'algorithme

Cet algorithme a un intérêt très important. Tout d'abord, il permet de faire l'analyse de tout mot pour toute grammaire hors contexte. Il est donc généraliste. De plus, sa complexité est assez facile à évaluer en $O(|x|^3.|G|)$. x étant le mot à analyser et $|G|$ le nombre de règles de la grammaire. Pour une grammaire donnée, on estime donc le coût de l'analyse en $O(|x|^3)$, ce qui donne une borne supérieure à la complexité du problème de l'analyse syntaxique dans le cas des grammaires hors-contexte.

Étonnement, cet algorithme (datant des années 60 [2]) est encore très utile de nos jours. En effet, il fait plus que l'analyse du mot complet, mais il fait une analyse de toutes les sous-chaînes de ce mot, ce qui fait qu'on peut obtenir des analyses partielles, même lorsque le mot n'est pas dans la grammaire. Cet avantage est très intéressant pour l'analyse des langues naturelles, où on peut être confronté à de nombreuses phrases agrammaticales, et aussi dans des cas ou l'on veut simplement repérer des syntagmes (groupes verbaux ou nominaux, etc.). Il est notamment encore utilisé dans les systèmes actuels de traduction automatique statistique.

2.4 Exercices

Exercice E3.16:

Mettre la grammaire des expressions sous FNC

$E \rightarrow E + T | T$
$T \rightarrow T * F | F$
$F \rightarrow (E) | a$

2. La date reste imprécise car si Kasami a publié cet algorithme dans un rapport en 1965 et Younger dans un article en 1968, Cocke ne semble pas avoir publié ce travail, qui a circulé de manière informelle. Il semble néanmoins que ce soit bien lui qui ait proposé cet algorithme dans un workshop (sur un coin de table) et que cet algorithme ait été utilisé notamment à Grenoble entre 61 et 70 dans le système traduction automatique "à règles" développé au CETA à cette époque. Des publications (en français) attestent cette antériorité.

Exercice E3.17:

Utilisez l'algorithme de Cocke ci-dessus pour déterminer si le mot « (aa)+a » fait parti du langage défini par la grammaire de l'exemple . Même question avec le mot « (a+a)*a ».